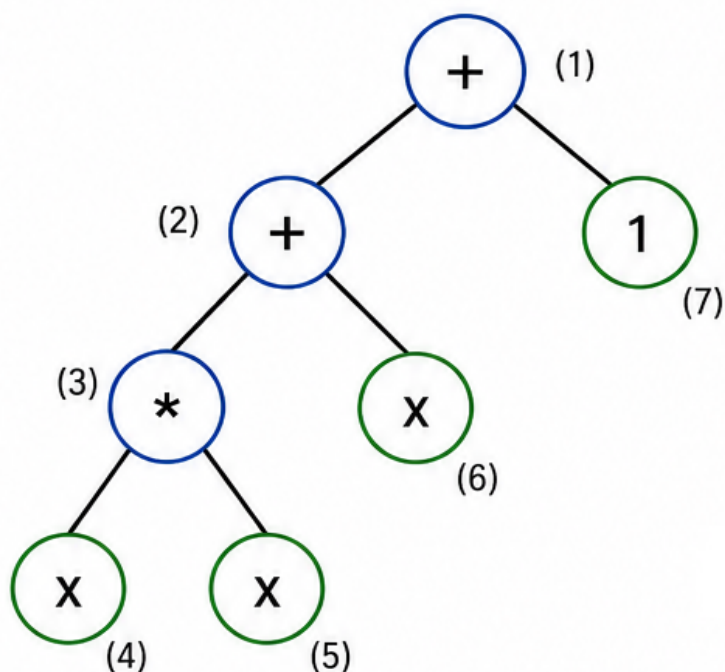


Breve introdução à Programação Genética e Regressão Simbólica em C++23



```
1 void mutate(node, p) {  
2     // subtree mutation (Koza)  
3     vector< Node* > nodes;  
4     collect_nodes(tree, nodes);  
5     auto* pt = nodes[rand()%  
6                 nodes.size()];  
7     *pt = random_tree(2);  
8 }
```

$$x^2 + x + 1$$

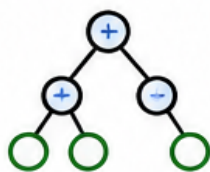
● Operadores

● Terminais (variáveis / constantes)

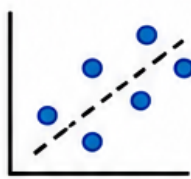
(n) Índice do nó



Algoritmos
Evolutivos



Árvores de
Expressão



Regressão
Simbólica



Implementação
em C++23



Modelos
Interpretáveis

Programação Genética e Regressão Simbólica

Breve Introdução

C++23

Este material foi elaborado a partir de anotações pessoais e de conteúdos consultados ao longo dos estudos, incluindo livros, artigos, documentações, vídeos e materiais disponíveis na internet.

Como as fontes não foram registradas durante a elaboração dessas anotações, não foi possível incluir referências bibliográficas completas.

Ferramentas de inteligência artificial foram utilizadas como suporte à revisão e ao aprimoramento do texto, bem como para auxiliar na identificação de possíveis erros em códigos e na elaboração de comentários e docstrings. Todas as contribuições geradas por essas ferramentas foram revisadas e validadas.

O conteúdo é disponibilizado exclusivamente para fins didáticos e educacionais.

O código-fonte relacionado a este material está disponível no repositório GitHub: https://github.com/mcleber/Genetic_Programming_and_Symbolic_Regression

Caso seja identificada a reprodução de algum trecho que deva receber atribuição específica, por favor entre em contato para que os devidos créditos sejam adicionados. Da mesma forma, se você encontrar erros, imprecisões ou oportunidades de melhoria, sintase à vontade para abrir uma *issue* no repositório. Contribuições e correções são sempre bem-vindas.

Cleber Moretti

Versão 1.0

2026

Sumário

I	PROGRAMAÇÃO GENÉTICA	3
1	O que é Programação Genética (PG)	3
1.1	Componentes principais	3
1.2	Operadores genéticos	3
1.3	Estrutura e Iteração	3
2	Exemplo de Programação Genética	4
2.1	A equação alvo	4
2.2	O que a programação genética faz	4
2.3	Como o código representa a equação	5
2.4	Resumo	5
3	Características do código	6
3.1	<i>Smart pointers</i> (<code>unique_ptr</code>)	6
3.2	Impressão hierárquica da árvore	6
3.3	Uso de C++23	6
4	Estrutura do projeto sugerida	7
5	Instruções de compilação e execução	7
6	Código C++: Programação Genética	8
7	CMakeLists	16
II	REGRESSÃO SIMBÓLICA	17
8	Regressão Simbólica com Programação Genética	17
8.1	Como a árvore representa uma equação	17
8.2	Cálculo de <i>Fitness</i> em Detalhe	17
8.3	Aplicações Práticas da Regressão Simbólica	18
9	O Problema do Bloat	19
9.1	Por que o <i>bloat</i> acontece?	19
9.2	Consequências práticas	19
9.3	Estratégias de controle de bloat	20
10	Visualização da Evolução do Algoritmo	21
10.1	Exportando dados para CSV	21
10.2	Visualizando com Python	22
10.3	O que observar no gráfico	23
11	Limitações da Implementação	24

12 Melhorias - Roteiro Prático	25
12.1 Funções Auxiliares - Já Implementadas	25
12.2 tree_size e Parsimony Pressure (anti-bloat)	25
12.3 Elitismo	26
12.4 Divisão Protegida	28
12.5 Múltiplas Variáveis	28
12.6 Paralelismo com <code>std::execution</code>	30
13 Conclusão	31

Parte I

PROGRAMAÇÃO GENÉTICA

1 O que é Programação Genética (PG)

Programação Genética (PG) é uma técnica de Computação Evolutiva que evolui programas ou expressões para resolver um problema. Ela se inspira na seleção natural e nos princípios da genética, aplicando-os sobre estruturas de código (geralmente árvores de expressões).

1.1 Componentes principais

- **Genótipo:** Representação interna do indivíduo. Na PG, normalmente é uma árvore de expressões (nós = operações, folhas = variáveis ou constantes).
- **Fenótipo:** A expressão ou programa “real” que é executado e avaliado.
- **Função de *fitness* (aptidão):** Mede o quão bom o programa é para resolver o problema.

1.2 Operadores genéticos

- **Crossover (Koza — *subtree crossover*):** Combina duas árvores escolhendo um ponto de corte aleatório em cada uma e trocando as subárvores inteiras entre os dois indivíduos.
- **Mutação (Koza — *subtree mutation*):** Seleciona um nó aleatório na árvore e substitui toda a subárvore enraizada naquele nó por uma nova subárvore gerada aleatoriamente, introduzindo diversidade estrutural na população.
- **Seleção:** mantém os melhores indivíduos da população (seleção por truncamento), preservando os de maior *fitness* para a próxima geração.

1.3 Estrutura e Iteração

- **População:** Conjunto de programas candidatos.
- **Iteração:** O processo é repetido até atingir um critério de parada, como atingir *fitness* suficiente ou número máximo de gerações.

2 Exemplo de Programação Genética

2.1 A equação alvo

No código, a função usada como referência (função alvo) é:

$$f(x) = x^2 + x + 1$$

Ela aparece no cálculo do *fitness*:

```
double y_true = x*x + x + 1;
```

- $x \cdot x \rightarrow$ termo quadrático
- $x \rightarrow$ termo linear
- $1 \rightarrow$ constante

Para cada valor de x , o programa calcula o resultado exato desta função.

2.2 O que a programação genética faz

O objetivo do algoritmo é evoluir árvores de expressões que se aproximem desta função.

Passos principais:

1. Criar população inicial de árvores aleatórias com operações $+$, $-$, $\cdot$ e variáveis x e constantes (1 a 5). Por exemplo, uma árvore inicial pode ser:

$$\begin{array}{c} + \\ / \backslash \\ x \quad 3 \end{array}$$

Que corresponde à expressão $x + 3$.

2. Avaliar *fitness*: Para cada árvore, o programa calcula:

$$\text{erro} = \sum_{x=-2}^2 |f_{\text{tree}}(x) - (x^2 + x + 1)|$$

onde:

- $f_{\text{tree}}(x) \rightarrow$ valor da expressão da árvore
- $f_{\text{alvo}}(x) \rightarrow$ valor da função $x^2 + x + 1$

O *fitness* é definido como o negativo do erro ($\text{fitness} = -\text{erro}$) para que maior *fitness* signifique melhor aproximação.

3. Evolução:

- **Seleção:** Mantém árvores com melhor *fitness*.
- **Crossover (Koza):** Troca subárvores inteiras entre dois indivíduos, escolhendo pontos de corte aleatórios em cada pai.
- **Mutação (Koza):** Seleciona um nó aleatório e substitui toda a subárvore a partir daquele nó por uma nova árvore gerada aleatoriamente.

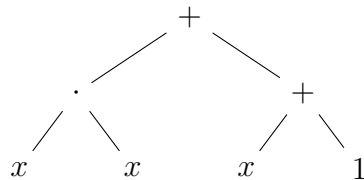
O processo se repete por várias gerações, permitindo que a árvore evolua.

2.3 Como o código representa a equação

O código não conhece a fórmula $x^2 + x + 1$. Ele apenas tenta aproximar usando:

- Operações básicas: $+$, $-$, $\cdot$
- Variável x e constantes pequenas (1 a 5)

Após algumas gerações, o algoritmo pode produzir árvores como:



Que corresponde exatamente a $x \cdot x + x + 1$, mostrando que a PG consegue descobrir ou evoluir a equação alvo automaticamente, sem conhecê-la a priori.

Observação: No código, o operador de multiplicação é representado por “*”, enquanto na notação matemática utilizamos “.”.

2.4 Resumo

- Equação alvo: $f(x) = x^2 + x + 1$
- Programa: aproxima a função usando árvores de operações.
- *Fitness*: soma do erro absoluto para $x = -2, -1, 0, 1, 2$
- Objetivo da PG: evoluir árvores até encontrar a expressão correta.

3 Características do código

3.1 *Smart pointers* (`unique_ptr`)

- Gerenciam automaticamente a memória dos nós da árvore.
- O `move()` é utilizado para transferir a propriedade de nós entre vetores (propriedade única), evitando cópias desnecessárias.

3.2 Impressão hierárquica da árvore

- Permite visualizar claramente a estrutura da árvore, com operadores e operandos.
- Os filhos direitos são impressos “acima” e os filhos esquerdos “abaixo”, facilitando a leitura.

3.3 Uso de C++23

Este subtópico apresenta três recursos utilizados no desenvolvimento, cada um com uma justificativa técnica específica.

Recurso	Onde é usado	Por que importa
<code>std::format</code>	Impressão das gerações	Evita mistura de <code>printf/cout</code> ; mais seguro e moderno para tipos fortemente tipados
<code>std::views::iota</code>	Loop de <i>fitness</i>	Gera sequência <i>lazy</i> sem alocar vetor; código mais limpo e eficiente
<code>std::unique_ptr</code>	Todos os nós da árvore	Gerência automática de memória; elimina necessidade de <code>delete</code> manual

Tabela 1: Recursos do C++ utilizados na implementação

4 Estrutura do projeto sugerida

```
Genetic_Programming_and_Symbolic_Regression/  
|  
+-- src/  
|   +-- CMakeLists.txt  
|   +-- main.cpp  
|   +-- plot_fitness.py  
|  
+-- docs/  
|   +-- programacao_genetica.pdf  
|  
+-- .editorconfig  
+-- .gitignore  
+-- License  
+-- README.md
```

5 Instruções de compilação e execução

No terminal:

```
cd programacao_genetica  
mkdir build && cd build  
cmake ..  
cmake --build .  
./programacao_genetica
```

para salvar a saída no diretório build/:

```
./programacao_genetica > output.txt
```

e para abrir o arquivo:

```
cat output.txt
```

Requer um compilador com suporte a C++23.

Testado com GCC 13+.

6 Código C++: Programação Genética

```
/**
 * @file main.cpp
 * @brief Implementacao de Programacao Genetica em C++23.
 *
 * Este programa evolui arvores de expressoes matematicas
 * para aproximar a funcao alvo:  $f(x) = x^2 + x + 1$ .
 *
 * Operadores de Koza implementados:
 * - Subtree crossover: troca subarvores inteiras entre dois pais.
 * - Subtree mutation: substitui uma subarvore por outra gerada
 *   aleatoriamente.
 *
 * - Usa std::unique_ptr para gerenciar nos da arvore.
 * - Suporta operadores binarios: +, -, *.
 * - Suporta variaveis (x) e constantes pequenas.
 * - Implementa: avaliacao, fitness, crossover, mutacao e impressao
 *   hierarquica.
 *
 * EXECUTAR
 * cd path/to/programacao_genetica
 * mkdir build
 * cd build
 * cmake ..
 * cmake --build .
 * ./programacao_genetica
 *
 * @author Cleber Moretti
 * @date 30 sep 2025
 */

#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <cstdlib>
#include <ctime>
#include <algorithm>
#include <cmath>
#include <format>
#include <ranges>

// -----
// Estrutura de no da arvore de expressao
// -----
/**
 * @struct Node
 * @brief Representa um no em uma arvore de expressao matematica.
 *
 * Cada no pode ser:
 * - Um operador binario: "+", "-", "*"
 * - Um operando: "x" ou uma constante (ex.: "1", "2").
 *
 * A arvore e gerenciada com std::unique_ptr para evitar vazamento de
 * memoria.
 */
struct Node
```

```

{
    std::string value;                ///< Valor do no ("+", "-",
        "x", "x" ou constante)
    std::unique_ptr<Node> left = nullptr; ///< Ponteiro inteligente
        para filho esquerdo
    std::unique_ptr<Node> right = nullptr; ///< Ponteiro inteligente
        para filho direito

    ///< Construtor
    explicit Node(std::string val) : value(std::move(val)) {}
};

// -----
// Avalia a arvore para um dado valor de x
// -----
/**
 * @brief Avalia a expressao representada pela arvore de expressao.
 *
 * Esta funcao percorre a arvore recursivamente:
 * - Se o no for folha, retorna o valor da constante ou da variavel "x".
 * - Se o no for operador, avalia os filhos esquerdo e direito e aplica
   o operador.
 * - Se houver valor invalido ou operador desconhecido, retorna 0 e
   exibe aviso.
 *
 * @param node Ponteiro para o no raiz da arvore.
 * @param x Valor da variavel "x" para avaliacao.
 * @return Resultado da avaliacao da expressao.
 */
double evaluate(const std::unique_ptr<Node> &node, double x)
{
    if (!node)
        return 0.0;

    if (!node->left && !node->right)
    {
        if (node->value == "x")
            return x;

        try
        {
            return std::stod(node->value);
        }
        catch (const std::invalid_argument &)
        {
            std::cerr << "Aviso: valor invalido '" << node->value
                << "' encontrado em no folha.\n";
            return 0.0;
        }
    }

    double l = evaluate(node->left, x);
    double r = evaluate(node->right, x);

    if (node->value == "+") return l + r;
    if (node->value == "-") return l - r;
    if (node->value == "*") return l * r;

```

```

std::cerr << "Aviso: operador invalido '" << node->value << "'
          encontrado.\n";
return 0.0;
}

// -----
// Cria arvore aleatoria
// -----
/**
 * @brief Gera uma arvore de expressao aleatoria.
 * @param depth Profundidade maxima da arvore (default = 2).
 * @return Ponteiro unico para a arvore gerada.
 */
std::unique_ptr<Node> random_tree(int depth = 2)
{
    if (depth == 0)
    {
        if (std::rand() % 2)
            return std::make_unique<Node>("x");

        return std::make_unique<Node>(std::to_string(std::rand() % 5 +
            1));
    }

    std::vector<std::string> ops = {"+", "-", "*"};
    auto node = std::make_unique<Node>(ops[std::rand() % ops.size()]);
    node->left = random_tree(depth - 1);
    node->right = random_tree(depth - 1);
    return node;
}

// -----
// Calcula fitness da arvore
// -----
/**
 * @brief Mede a qualidade da arvore em aproximar  $f(x) = x^2 + x + 1$ .
 * @param tree Ponteiro para a arvore de expressao.
 * @return Fitness (quanto maior, melhor; 0 = solucao exata).
 */
double fitness(const std::unique_ptr<Node> &tree)
{
    double err = 0.0;
    for (double x : std::views::iota(-5, 6))
    {
        double y_true = x * x + x + 1.0;
        double y_pred = evaluate(tree, x);
        err += std::abs(y_true - y_pred);
    }
    return -err;
}

// -----
// CLONE -- copia profunda e independente de uma arvore
// -----
/**
 * @brief Cria uma copia completa e independente da arvore.
 *
 * Necessario para elitismo, crossover e mutacao de Koza:

```

```

* std::unique_ptr nao permite copia implicita, entao toda duplicacao
* de arvore passa obrigatoriamente por esta funcao.
*
* @param node Raiz da arvore a ser clonada.
* @return Nova arvore independente com a mesma estrutura e valores.
*/
std::unique_ptr<Node> clone_tree(const std::unique_ptr<Node>& node)
{
    if (!node) return nullptr;

    auto copy = std::make_unique<Node>(node->value);
    copy->left = clone_tree(node->left);
    copy->right = clone_tree(node->right);
    return copy;
}

// -----
// COLETA DE NOS -- necessario para crossover e mutacao de Koza
// -----
/**
* @brief Coleta ponteiros para todos os unique_ptr de nos da arvore.
*
* Permite escolher um no aleatorio sem conhecer a estrutura com
* antecendencia -- base do subtree crossover e subtree mutation de Koza.
*
* @param node No atual (passar a raiz na chamada inicial).
* @param nodes Vetor que recebera os ponteiros coletados.
*/
void collect_nodes(
    std::unique_ptr<Node>& node,
    std::vector<std::unique_ptr<Node>*>& nodes)
{
    if (!node) return;

    nodes.push_back(&node);
    collect_nodes(node->left, nodes);
    collect_nodes(node->right, nodes);
}

// -----
// CROSSOVER DE KOZA -- subtree crossover
// -----
/**
* @brief Subtree crossover classico de Koza (1992).
*
* Algoritmo:
* 1. Clona o pai A -- esse clone se tornara o filho.
* 2. Coleta todos os nos do filho (clone de A) e do pai B.
* 3. Escolhe um ponto de corte aleatorio no filho.
* 4. Escolhe uma subarvore doadora aleatoria de B.
* 5. Substitui a subarvore no ponto de corte pelo clone da doadora.
*
* @param a Primeiro pai.
* @param b Segundo pai.
* @return Novo individuo resultante do crossover.
*/
std::unique_ptr<Node> crossover(
    const std::unique_ptr<Node>& a,

```

```

    const std::unique_ptr<Node>& b)
{
    if (!a || !b) return nullptr;

    // Passo 1: filho = clone de A
    auto child = clone_tree(a);

    // Passo 2: coleta nos do filho e de B
    std::vector<std::unique_ptr<Node>*> child_nodes;
    collect_nodes(child, child_nodes);

    // Passos 3-5: substitui subarvore escolhida
    auto donor = clone_tree(b);
    std::vector<std::unique_ptr<Node>*> donor_nodes;
    collect_nodes(donor, donor_nodes);

    auto* cut_point =
        child_nodes[std::rand() % child_nodes.size()];

    if (donor_nodes.empty())
        return child;

    // tenta achar um no interno
    std::unique_ptr<Node>* donor_point = nullptr;

    bool found = false;

    for (int attempt = 0; attempt < 10; ++attempt)
    {
        auto* candidate =
            donor_nodes[std::rand() % donor_nodes.size()];

        if ((*candidate)->left || (*candidate)->right)
        {
            donor_point = candidate;
            found = true;
            break;
        }
    }

    // fallback: aceita qualquer no se nao achou interno
    if (!found)
    {
        donor_point =
            donor_nodes[std::rand() % donor_nodes.size()];
    }

    *cut_point = clone_tree(*donor_point);

    return child;
}

// -----
// MUTACAO DE KOZA -- subtree mutation
// -----
/**
 * @brief Subtree mutation classica de Koza (1992).
 *

```

```

* Algoritmo:
* 1. Coleta todos os nos da arvore.
* 2. Escolhe um no aleatorio como ponto de mutacao.
* 3. Substitui a subarvore inteira enraizada naquele no por uma
*    nova subarvore gerada aleatoriamente com random_tree().
*
* Diferenca em relacao a point mutation:
* - Point mutation: apenas troca o valor de um no (folha ou operador),
*   mantendo a estrutura topologica da arvore inalterada.
* - Subtree mutation (Koza): substitui a subarvore inteira a partir do
*   ponto escolhido, alterando estrutura e valores simultaneamente.
*   Introduz maior diversidade estrutural na populacao.
*
* @param tree Arvore a ser mutada (modificada in-place).
* @param prob Probabilidade de ocorrer a mutacao (default = 40%).
*/
void mutate(std::unique_ptr<Node>& tree, double prob = 0.40)
{
    if (!tree) return;

    // Aplica mutacao apenas se o sorteio for favoravel
    if ((std::rand() / static_cast<double>(RAND_MAX)) >= prob)
        return;

    // Coleta todos os nos para escolher o ponto de mutacao
    std::vector<std::unique_ptr<Node>*> nodes;
    collect_nodes(tree, nodes);

    // Escolhe um no aleatorio e substitui sua subarvore por uma nova
    auto* mutation_point = nodes[std::rand() % nodes.size()];
    *mutation_point = random_tree(2);
}

// -----
// Converte arvore em string legivel
// -----
/**
 * @brief Converte arvore em expressao legivel (notacao infixa com
 *   parenteses).
 */
std::string tree_to_string(const std::unique_ptr<Node> &node)
{
    if (!node) return "";
    if (!node->left && !node->right) return node->value;

    return "(" +
        tree_to_string(node->left) +
        node->value +
        tree_to_string(node->right) +
        ")";
}

// -----
// Imprime arvore hierarquica
// -----
/**
 * @brief Imprime arvore em formato hierarquico (rotacionada 90 graus).
 */

```

```

    * Filhos diretos aparecem acima, filhos esquerdos abaixo,
    * com recuo proporcional a profundidade do no.
    */
void print_tree(const std::unique_ptr<Node> &node, int level = 0)
{
    if (!node) return;

    print_tree(node->right, level + 1);
    std::cout << std::string(level * 4, ' ') << node->value << "\n";
    print_tree(node->left, level + 1);
}

// -----
// Funcao principal
// -----
int main()
{
    std::srand(std::time(nullptr));

    const int POP_SIZE      = 200; // mais diversidade
    const int GENERATIONS  = 100; // mais tempo para evoluir

    // Cria populacao inicial com arvores aleatorias
    std::vector<std::unique_ptr<Node>> population;
    for (int i = 0; i < POP_SIZE; i++)
        population.push_back(random_tree());

    for (int gen = 0; gen < GENERATIONS; gen++)
    {
        // Avalia fitness de cada individuo
        std::vector<std::pair<double, int>> scored;
        for (int i = 0; i < POP_SIZE; i++)
            scored.push_back({fitness(population[i]), i});

        // Ordena do melhor para o pior
        std::sort(scored.rbegin(), scored.rend());

        // Exibe melhor individuo da geracao
        std::cout << std::format(
            "Geracao {:2}: Melhor fitness = {:.2f}\n",
            gen, scored[0].first);
        std::cout << "Expressao: "
            << tree_to_string(population[scored[0].second]) <<
            "\n";
        std::cout << "Arvore hierarquica:\n";
        print_tree(population[scored[0].second]);
        std::cout << std::string(60, '-') << "\n";

        // Selecao por truncamento: mantem o top 50%
        std::vector<std::unique_ptr<Node>> new_pop;
        for (int i = 0; i < POP_SIZE / 2; i++)
            new_pop.push_back(clone_tree(population[scored[i].second]));

        // Gera novos filhos por crossover + mutacao de Koza
        while (new_pop.size() < static_cast<std::size_t>(POP_SIZE))
        {
            auto& a = new_pop[std::rand() % (POP_SIZE / 2)];
            auto& b = new_pop[std::rand() % (POP_SIZE / 2)];

```



```

        auto child = crossover(a, b); // subtree crossover de Koza
        mutate(child);               // subtree mutation de Koza

        new_pop.push_back(std::move(child));
    }

    population = std::move(new_pop);
}

return 0;
}

```

7 CMakeLists

Criar o arquivo CMakeLists.txt na raiz do projeto.

```
# -----  
# CMake minimo necessario  
# -----  
cmake_minimum_required(VERSION 3.25) # Versao minima do CMake  
# -----  
# Nome e versao do projeto  
# -----  
project(ProgramacaoGenetica  
    VERSION 1.0  
    LANGUAGES CXX  
)  
# -----  
# Configuracao do padrao C++  
# -----  
set(CMAKE_CXX_STANDARD 23)           # Usar C++23  
set(CMAKE_CXX_STANDARD_REQUIRED ON)  # Exigir padrao C++23  
set(CMAKE_CXX_EXTENSIONS OFF)        # Desativar extensoes especificas  
    do compilador  
# -----  
# Adiciona executavel  
# -----  
add_executable(programacao_genetica src/main.cpp)  
# -----  
# Mensagens informativas  
# -----  
message(STATUS "Projeto: ${PROJECT_NAME}")  
message(STATUS "Versao C++: ${CMAKE_CXX_STANDARD}")  
message(STATUS "Fonte: main.cpp")
```

Parte II

REGRESSÃO SIMBÓLICA

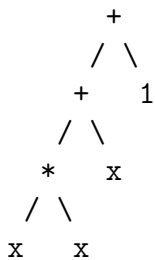
8 Regressão Simbólica com Programação Genética

A regressão simbólica é a aplicação mais clássica da Programação Genética. Seu objetivo é encontrar automaticamente uma expressão matemática, tanto sua estrutura quanto seus parâmetros, capaz de representar um conjunto de dados. Diferente da regressão linear ou polinomial, não assumimos nenhuma forma prévia para a equação: o algoritmo parte do zero e descobre por conta própria.

No tutorial, a função alvo é $f(x) = x^2 + x + 1$. O algoritmo não conhece essa expressão. Ele apenas recebe pares $(x, f(x))$ e tenta evoluir árvores que produzam os mesmos valores. Isso simula o cenário real: dados observados, lei desconhecida.

8.1 Como a árvore representa uma equação

Cada nó interno da árvore é um operador (+, -, *) e cada folha é uma variável (x) ou constante numérica. A avaliação segue a travessia em pós-ordem: avalia os filhos recursivamente e aplica o operador da raiz. A expressão $x^2 + x + 1$ em árvore:



Árvore correspondente a $(x \times x) + x + 1$.

A função `evaluate()` percorre essa estrutura recursivamente. Quando chega a um nó folha com valor "x", retorna o valor da variável. Para constantes, converte a *string* para `double`. Para operadores, avalia ambos os filhos e aplica a operação.

8.2 Cálculo de *Fitness* em Detalhe

O fitness mede o quão bem uma árvore aproxima a função alvo. No código, ele é calculado somando o erro absoluto para cinco valores de x :

```
double fitness(const std::unique_ptr<Node>& tree)
{
    double err = 0.0;
    for (double x : std::views::iota(-5, 6)) // x in {-5,-3,-2,...,5}
    {
        double y_true = x * x + x + 1; // funcao alvo real
        double y_pred = evaluate(tree, x); // saida da arvore
        err += std::abs(y_true - y_pred); // erro absoluto
    }
    return -err; // negativo: maior fitness = menor erro
}
```

O *fitness* é o negativo do erro total. Como o algoritmo maximiza *fitness*, **fitness** = 0 significa erro zero, a árvore encontrou a equação exata. Quanto mais negativo, pior a aproximação.

Nota

Por que apenas 5 pontos de x ? É suficiente para distinguir expressões simples, mas em problemas reais você usaria dezenas ou centenas de amostras. Com poucos pontos, corre-se o risco de *overfitting*: a árvore acerta os 5 pontos, mas falha fora deles.

8.3 Aplicações Práticas da Regressão Simbólica

A regressão simbólica não é apenas um exercício acadêmico:

- **Engenharia e física experimental:** descobrir equações de atrito, coeficientes de arrasto ou relações de materiais a partir de dados de ensaio, sem modelo teórico prévio.
- **Bioinformática:** modelar expressão gênica e interações proteicas onde a forma da equação é completamente desconhecida.
- **Finanças quantitativas:** buscar indicadores técnicos ou regras de trading que combinam preço, volume e outros dados de forma não-linear.
- **Modelagem climática:** encontrar parametrizações simples para fenômenos como convecção ou albedo a partir de simulações de alta resolução.
- **Descoberta automática de leis físicas:** o projeto AI Feynman usa variantes de regressão simbólica para redescobrir equações do livro de Feynman a partir de dados experimentais.

9 O Problema do Bloat

Bloat (literalmente, “inchaço”) é o crescimento descontrolado das árvores ao longo das gerações. Com o tempo, as árvores acumulam subárvores que não contribuem para o *fitness*, código morto genético. Uma árvore que deveria representar $x+1$ pode evoluir para algo como $((x \times 1) + (3 - 2)) + (x \times 0 + 1)$, semanticamente equivalente, mas estruturalmente inflada.

Atenção

O *bloat* é o principal problema prático da Programação Genética clássica. Entendê-lo é fundamental antes de qualquer aplicação séria.

9.1 Por que o *bloat* acontece?

- **Pressão de crescimento por *crossover*:** quando duas árvores trocam subárvores, é estatisticamente mais provável que o filho seja maior do que menor, a maioria dos nós está em subárvores profundas, então o *crossover* tende a inserir material, não remover.
- **Subárvores neutras (introns):** subexpressões como $x \times 1$, $x + 0$ ou $a - a$ não alteram o valor da expressão, mas ocupam espaço. Por não prejudicarem o *fitness*, não são eliminadas pela seleção. Em termos teóricos, essas subárvores funcionam como regiões não-codificantes do DNA: protegem o material genético ao redor de mutações sem contribuir para a expressão final. Esse comportamento pode ser observado diretamente na saída do programa, onde expressões como $x \times 1$ e $a - a$ aparecem com frequência nas árvores evoluídas.

A implementação atual não inclui controle de *bloat*. Estratégias de mitigação como *parsimony pressure*, limite máximo de profundidade e *crossover* com restrição de tamanho são abordadas na seção de melhorias.

9.2 Consequências práticas

No código implementado anteriormente, a profundidade inicial é 2 e não há controle de tamanho máximo. Em execuções longas ou com população maior, é comum ver árvores com dezenas de nós, consumindo mais memória, mais tempo de avaliação e dificultando a interpretação da expressão evoluída.

Geração	Nós médios	<i>Fitness</i> médio	Observação
0	7	-28,4	Árvores iniciais, profundidade 2
5	12	-15,1	<i>Crossover</i> começa a crescer estruturas
10	19	-8,3	<i>Bloat</i> visível; algumas subárvores inúteis
20	31	-3,7	Árvores infladas; avaliação mais lenta

Tabela 2: Evolução típica do tamanho das árvores (valores ilustrativos)

9.3 Estratégias de controle de bloat

- **Limite de profundidade máxima (*hard cap*):** rejeita ou poda qualquer árvore que exceda um tamanho definido. Simples de implementar, mas pode descartar estruturas válidas que cresceram legitimamente.
- **Penalização no *fitness* (*parsimony pressure*):** subtrai do *fitness* um valor proporcional ao tamanho da árvore: $\text{fitness_ajustado} = \text{fitness} - \alpha \times \text{tamanho}$. Favorece expressões menores sem descartá-las abruptamente.
- ***Crossover* por subárvore com restrição de tamanho:** escolhe pontos de corte aleatórios nas duas árvores e limita o tamanho do filho. É o *crossover* padrão em implementações modernas como DEAP e gplearn.

10 Visualização da Evolução do Algoritmo

Acompanhar o *fitness* geração a geração é essencial para diagnosticar o comportamento do algoritmo. Dois fenômenos importantes são visíveis em um gráfico: convergência saudável (*fitness* cresce suavemente até estabilizar) e convergência prematura (*fitness* estagna cedo, indicando que a população perdeu diversidade genética).

10.1 Exportando dados para CSV

Adicione o bloco abaixo ao `main()`. Ele grava um arquivo CSV com melhor *fitness*, *fitness* médio e tamanho médio por geração. O tamanho médio é o indicador do *bloat*, se cresce enquanto o *fitness* estagna, o *bloat* está dominando.

```
#include <fstream>

// --- Antes da funcao fitness ---
// -----
// Conta o numero de nos de uma arvore
// -----
/**
 * @brief Conta recursivamente o numero total de nos de uma arvore.
 *
 * Esta funcao percorre toda a arvore e retorna a quantidade de
 * nos que ela contem, incluindo tanto os nos internos (operadores)
 * quanto os nos terminais (variaveis e constantes).
 *
 * Essa metrica e utilizada para monitorar o crescimento das arvores
 * ao longo do processo evolutivo e para analisar o fenomeno de
 * bloat (crescimento excessivo de codigo) em Programacao Genetica.
 *
 * @param node No raiz da arvore.
 * @return Numero total de nos da arvore.
 */
std::size_t tree_size(const std::unique_ptr<Node>& node)
{
    if (!node)
        return 0;

    return 1
        + tree_size(node->left)
        + tree_size(node->right);
}

// --- Dentro de main(), antes do loop de evolucao: ---
std::ofstream csv("evolucao_fitness.csv");
csv << "geracao,melhor_fitness,fitness_medio,tamanho_medio\n";

// --- Dentro do loop, apos calcular scored: ---
double sum_fit = 0.0;
double sum_size = 0.0;
for (auto& [f, i] : scored)
{
    sum_fit += f;
    sum_size += tree_size(population[i]);
}
double avg_fit = sum_fit / POP_SIZE;
```

```
double avg_size = sum_size / POP_SIZE;

csv << std::format("{},{:.4f},{:.4f},{:.1f}\n",
    gen, scored[0].first, avg_fit, avg_size);

// --- Apos o loop de evolucao: ---
csv.close();
```

10.2 Visualizando com Python

É necessário criar um ambiente virtual **venv** para isolar as dependências do projeto e evitar conflitos com pacotes globais do sistema.

Em algumas distribuições, o **venv** já vem incluído com o Python. Entretanto, havendo a necessidade o comando para instalar é:

```
sudo apt install python3-venv
```

Para criar um ambiente virtual, utiliza-se o comando:

```
python3 -m venv venv
```

Este comando cria um diretório chamado **venv**, contendo um ambiente isolado do Python, permitindo a instalação de dependências sem interferir no sistema global.

Para ativar o ambiente virtual, no Linux, utiliza-se:

```
source venv/bin/activate
```

Para sair do ambiente virtual, basta executar:

```
deactivate
```

A atualização do gerenciador de pacotes **pip** pode ser realizada com o ambiente ativo por meio do comando:

```
python -m pip install --upgrade pip
```

Com o ambiente virtual ativado, é necessário instalar as dependências do projeto, especificamente as bibliotecas **pandas** e **matplotlib**, utilizando:

```
pip install pandas matplotlib
```

Essas bibliotecas são utilizadas para leitura de dados experimentais em formato CSV e para a geração de gráficos que representam a evolução do *fitness* e o crescimento médio das árvores ao longo das gerações.

Salve o *script* abaixo como `plot_fitness.py` na mesma pasta do executável e rode com `python3 plot_fitness.py` após executar o programa C++.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("evolucao_fitness.csv")
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

# Painel 1 - Fitness
ax1.plot(df["geracao"], df["melhor_fitness"], "b-o",
        label="Melhor fitness", linewidth=2, markersize=4)
ax1.plot(df["geracao"], df["fitness_medio"], "b--",
        alpha=0.5, label="Fitness medio")
ax1.axhline(0, color="green", linestyle=":", linewidth=1.5,
        label="Fitness ideal (erro = 0)")
ax1.set_ylabel("Fitness")
ax1.legend()
ax1.set_title("Evolucao do Fitness por Geracao")
ax1.grid(True, alpha=0.3)

# Painel 2 - Tamanho medio (indicador de bloat)
ax2.plot(df["geracao"], df["tamanho_medio"], "r-s",
        label="Tamanho medio das arvores", markersize=4)
ax2.set_xlabel("Geracao")
ax2.set_ylabel("Nos por arvore")
ax2.legend()
ax2.set_title("Crescimento das Arvores (Monitor de Bloat)")
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("evolucao.png", dpi=150)
plt.show()
```

10.3 O que observar no gráfico

- **Queda rápida nas primeiras gerações:** normal e esperada. A seleção elimina rapidamente os piores indivíduos.
- **Platô antes do *fitness* 0:** convergência prematura, a população ficou homogênea e o *crossover* não gera novidade. Solução: aumentar taxa de mutação, aumentar tamanho da população ou introduzir indivíduos aleatórios novos.
- **Oscilações bruscas:** mutação alta demais ou *crossover* destruindo estruturas boas. Reduza a taxa de mutação ou implemente elitismo.
- ***Fitness* = 0 atingido:** a árvore encontrou a equação exata ou uma expressão algebricamente equivalente.

11 Limitações da Implementação

A implementação é intencionalmente simples para fins didáticos. Entender suas limitações é o primeiro passo para superá-las.

População muito pequena. Com apenas 10 indivíduos, a diversidade genética é baixíssima e a convergência prematura é quase garantida. Para problemas reais, populações de 100 a 1000 indivíduos são comuns.

Sem controle de *bloat*. Não há limite de tamanho nem penalização para árvores grandes. Em execuções longas, as árvores crescem indefinidamente, consumindo memória e tempo.

Sem divisão protegida. O tutorial suporta apenas $+$, $-$, $*$. A divisão causaria divisão por zero quando o denominador evoluir para 0.

Constantes fixas (1 a 5). Expressões que precisem de coeficientes como 0,5 ou π não podem ser representadas. Solução: usar *ephemeral random constants* — constantes aleatórias de ponto flutuante geradas na criação do nó.

Sem memória entre execuções. Cada execução começa do zero. Para experimentação seria útil serializar a população para arquivo e retomar de onde parou.

12 Melhorias - Roteiro Prático

12.1 Funções Auxiliares - Já Implementadas

As duas funções abaixo já estão presentes no `main.cpp`. São listadas aqui como referência, pois são pré-requisito de todas as melhorias seguintes.

clone_tree — cópia profunda e independente de uma árvore

O C++ não copia `unique_ptr` automaticamente, é preciso fazer uma travessia recursiva criando novos nós. Sem isso, não é possível preservar o melhor indivíduo (elitismo) nem fazer *crossover* sem destruir os pais.

```
std::unique_ptr<Node> clone_tree(const std::unique_ptr<Node>& node)
{
    if (!node) return nullptr;
    auto copy = std::make_unique<Node>(node->value);
    copy->left = clone_tree(node->left); // clona filho esquerdo
    copy->right = clone_tree(node->right); // clona filho direito
    return copy;
}
```

collect_nodes — coleta ponteiros para todos os nós

Necessária para o *crossover* por subárvore: precisamos escolher um nó aleatório dentro de uma árvore sem saber sua estrutura com antecedência.

```
void collect_nodes(
    std::unique_ptr<Node>& node,
    std::vector<std::unique_ptr<Node>*>& nodes)
{
    if (!node) return;
    nodes.push_back(&node); // registra este nó
    collect_nodes(node->left, nodes); // desce a esquerda
    collect_nodes(node->right, nodes); // desce a direita
}
```

12.2 tree_size e Parsimony Pressure (anti-bloat)

A *parsimony pressure* é a estratégia mais eficaz contra o *bloat* sem descartar árvores abruptamente. Ela subtrai do *fitness* um valor proporcional ao número de nós da árvore, obtido pela função `tree_size`, favorecendo expressões menores com o mesmo poder preditivo. Dessa forma, o algoritmo passa a penalizar explicitamente estruturas mais complexas, incentivando soluções mais compactas e eficientes, em consonância com o princípio da navalha de Occam aplicado à evolução.

Para implementar será necessário substituir a função *fitness* atual:

```
double fitness(const std::unique_ptr<Node> &tree)

por:

/**
 * @brief Calcula o fitness de uma arvore de expressao com parsimony
 *        pressure.
 *
 * Esta funcao avalia quao bem a arvore de expressao dada aproxima a
 * funcao
 * alvo  $f(x) = x^2 + x + 1$  em um conjunto discreto de valores de
 * entrada.
 *
 * O fitness e definido como o negativo da soma dos erros absolutos
 * entre os
 * valores preditos e os valores reais, combinado com uma penalidade
 * estrutural
 * proporcional ao tamanho da arvore.
 *
 * O termo de penalidade implementa parsimony pressure, desencorajando
 * o crescimento
 * excessivo das arvores (bloat) ao penalizar estruturas maiores
 * atraves da metrica
 * tree_size.
 *
 * @param tree No raiz da arvore de expressao.
 * @param alpha Coeficiente que controla a intensidade da parsimony
 *        pressure (padrao = 0.03).
 *        Valores maiores aumentam a penalizacao para arvores
 *        grandes.
 * @return Valor de fitness (quanto maior, melhor). O valor maximo e 0
 *        para uma solucao
 *        perfeita e minima.
 */
double fitness(const std::unique_ptr<Node>& tree, double alpha = 0.03)
{
    double err = 0.0;

    for (double x : std::views::iota(-5, 6))
    {
        double y_true = x * x + x + 1.0;
        double y_pred = evaluate(tree, x);
        err += std::abs(y_true - y_pred);
    }

    double penalty = alpha * static_cast<double>(tree_size(tree));

    return -err - penalty; // penaliza arvores grandes (bloat control)
}
```

12.3 Elitismo

O elitismo garante que o melhor indivíduo de cada geração seja copiado intacto para a próxima, sem sofrer *crossover* ou mutação. Isso impede que o *fitness* decraça entre gerações e estabiliza a evolução.

Localizar no main():

```
// Truncation selection / Selecao por truncamento - mantem o top 50%:
std::vector<std::unique_ptr<Node>> new_pop;

// Preenchimento em 30%
int top_k = static_cast<int>(POP_SIZE * 0.3);
for (int i = 0; i < top_k; i++)
    new_pop.push_back(clone_tree(population[scored[i].second]));

while (new_pop.size() < POP_SIZE)
{
    const auto& a = population[std::rand() % POP_SIZE];
    const auto& b = population[std::rand() % POP_SIZE];

    auto child = crossover(a, b);
    mutate(child);

    new_pop.push_back(std::move(child));
}
```

E substituir o bloco todo por:

```
/**
 * @brief Estrategia de selecao com elitismo e reproducao da populacao.
 *
 * Este bloco implementa a construcao da nova populacao em tres etapas:
 *
 * 1. Elitismo: os ELITE_N melhores individuos sao copiados sem
 *    modificacao
 *    para garantir preservacao das melhores solucoes encontradas.
 *
 * 2. Reproducao parcial: o restante da nova populacao e preenchido com
 *    os
 *    melhores individuos do top 50% da geracao atual, garantindo
 *    pressao
 *    seletiva e manutencao de qualidade genetica.
 *
 * 3. Geracao de novos individuos: a populacao e completada atraves de
 *    crossover (Koza subtree crossover) seguido de mutacao, ate atingir
 *    o tamanho original da populacao.
 *
 * Esta estrategia combina exploracao (diversidade genetica) e
 * exploracao
 * (preservacao de solucoes boas), reduzindo degradacao de fitness
 * entre geracoes.
 */
constexpr int ELITE_N = 1; // quantos melhores preservar intactos
std::vector<std::unique_ptr<Node>> new_pop;

// 1. Copia os ELITE_N melhores sem modificacao
for (int i = 0; i < ELITE_N; i++)
    new_pop.push_back(clone_tree(population[scored[i].second]));

// 2. Completa com o restante do top 50%
for (int i = 0; i < POP_SIZE / 2; i++)
    new_pop.push_back(clone_tree(population[scored[i].second]));

// 3. Crossover + mutacao ate repor a populacao
while (new_pop.size() < static_cast<std::size_t>(POP_SIZE))
```

```

{
    auto& a = population[rand() % POP_SIZE];
    auto& b = population[rand() % POP_SIZE];
    auto child = crossover(a, b);
    mutate(child);
    new_pop.push_back(std::move(child));
}
population = std::move(new_pop);

```

12.4 Divisão Protegida

A divisão protegida permite adicionar o operador `/` ao conjunto de operadores sem o risco de divisão por zero. Quando o denominador é próximo de zero, a função retorna o próprio numerador (1), tratando a divisão como identidade, um valor neutro que não quebra a evolução.

```

// Dentro de double evaluate(const std::unique_ptr<Node> &node, double
// x), apos o operador '*':
if (std::abs(r) < 1e-9)
    return 1; // identidade neutra
return 1 / r;

// Em random_tree(), substitua o vetor de operadores:
std::vector<std::string> ops = {"+", "-", "*", "/"};

```

12.5 Múltiplas Variáveis

Para funções com mais de uma variável, como $f(x, y) = x^2 + 2xy + y^2$, substitua o parâmetro `double x` por um mapa de variáveis em todas as funções que recebem `x`.

Adicionar:

```
#include <unordered_map>
```

E logo abaixo dos *includes*:

```
using VarMap = std::unordered_map<std::string, double>;
```

Substituir completamente a função:

```
double evaluate(const std::unique_ptr<Node> &node, double x)
```

Pela função:

```

/**
 * @brief Avalia a arvore de expressao usando multiplas variaveis.
 *
 * Esta versao substitui a abordagem de uma unica variavel (x)
 * por um mapa de variaveis (VarMap), permitindo expressoes como:
 * f(x, y) = x*x + 2*x*y + y*y
 *
 * @param node Raiz da arvore de expressao
 * @param vars Mapa contendo valores das variaveis (ex: {"x":1.0,
 *              "y":2.0})
 * @return Resultado numerico da avaliacao
 */
double evaluate(const std::unique_ptr<Node>& node, const VarMap& vars)
{

```

```

if (!node)
    return 0.0;

// No folha (variavel ou constante)
if (!node->left && !node->right)
{
    // Verifica se eh uma variavel conhecida
    auto it = vars.find(node->value);
    if (it != vars.end())
        return it->second;

    // Caso contrario tenta converter para numero
    try
    {
        return std::stod(node->value);
    }
    catch (...)
    {
        return 0.0;
    }
}

double l = evaluate(node->left, vars);
double r = evaluate(node->right, vars);

if (node->value == "+") return l + r;
if (node->value == "-") return l - r;
if (node->value == "*") return l * r;

// Protecao contra divisao por zero (ou valor muito pequeno)
if (node->value == "/")
    return (std::abs(r) < 1e-9) ? 1.0 : (l / r);

return 0.0;
}

```

E dentro da função *fitness*, substituir:

```

std::vector<double> test = {
    -5, -3, -2, -1, -0.5, 0, 0.5, 1, 2, 3, 5};

for (double x : test)
{
    double y_true = x * x + x + 1.0;
    double y_pred = evaluate(tree, x);
    err += std::abs(y_true - y_pred);
}

```

Por:

```

for (double x : std::views::iota(-5, 6))
{
    // Crie um mapa de variaveis para esta avaliacao
    VarMap vars = { {"x", x} };

    double y_true = x * x + x + 1.0;
    double y_pred = evaluate(tree, vars);

    err += std::abs(y_true - y_pred);
}

```

12.6 Paralelismo com `std::execution`

O cálculo de *fitness* é o gargalo computacional e é trivialmente paralelizável: cada indivíduo é completamente independente dos outros. Com a STL paralela do C++17/23, uma única alteração distribui o trabalho pelos núcleos disponíveis.

Substituir:

```
std::vector<std::pair<double, int>> scored;

for (int i = 0; i < POP_SIZE; i++)
    scored.push_back({fitness(population[i]), i});
```

Por:

```
#include <execution>
#include <numeric>

// Vetor que armazena pares (fitness, indice) para cada individuo
std::vector<std::pair<double, int>> scored(POP_SIZE);

// Vetor de indices auxiliar usado para paralelizar a avaliacao
std::vector<int> indices(POP_SIZE);

// Preenche indices com valores de 0 ate POP_SIZE - 1
std::iota(indices.begin(), indices.end(), 0);

// Avaliacao paralela do fitness de cada individuo
std::for_each(
    std::execution::par_unseq, // execucao paralela + vetorizada (SIMD)
    indices.begin(), indices.end(),
    [&](int i) {
        // Calcula fitness do individuo i e armazena o resultado
        scored[i] = {fitness(population[i]), i};
    }
);

// Ordena individuos por fitness em ordem decrescente (melhores primeiro)
std::sort(scored.rbegin(), scored.rend());
```

Nota Linux

No Linux, `std::execution::par_unseq` requer Intel TBB (sudo apt install libtbb-dev). No Windows com MSVC e no macOS com Clang 16+, o suporte é nativo. Sem TBB, o código compila mas roda sequencialmente sem erro.

Adicionar no CMakeLists.txt:

```
# -----
# Biblioteca de paralelismo TBB
# -----
find_package(TBB REQUIRED)
target_link_libraries(programacao_genetica PRIVATE TBB::tbb)
```


13 Conclusão

O projeto cobre todos os componentes essenciais de um sistema de Programação Genética funcional. Com as melhorias deste material, o mesmo núcleo de código passa a ser competitivo para problemas reais de regressão simbólica.

O caminho recomendado: comece pelas funções auxiliares, ative a *parsimony pressure* e o elitismo, exporte o CSV e analise o gráfico. Só então avance para *crossover* real e divisão protegida. Múltiplas variáveis e paralelismo são o passo final para problemas de maior escala.